

リモセン応用セミナー WEBGIS実装講座

衛星データ解析技術研究会 技術セミナー（応用編）

第2回 タイル配信②

自前配信とカートグラフィック

2026年6月26日（金）

到達目標：地図配信環境を自前で用意(ホスティング)できるようになる。/ 地図データのスタイルの編集(カートグラフィック)を理解する。

タイムライン

時間	内容	目的
0:00–0:10	前回成果物の確認	PMTiles が手元にあることを確認
0:10–0:30	Nginx 静的配信・CORS	タイルサーバーなしで PMTiles を公開する
0:30–0:50	Maputnik スタイル設計	地図の見た目を GUI で編集・エクスポートする
0:50–1:00	— 中休み —	
1:00–1:15	MapLibre Style Spec	スタイル JSON の構造とコードとの対応を理解する
1:15–1:35	デモ①：ラズパイ接続	オフライン配信のユースケースを体感する
1:35–1:55	OSM・iD エディタ	自分でデータを整備してパイプラインに乗せる
1:55–2:20	デモ②：地理院地図ベクター	スタイルカスタマイズの実例を見る
2:20–2:30	まとめ	

ガイダンス

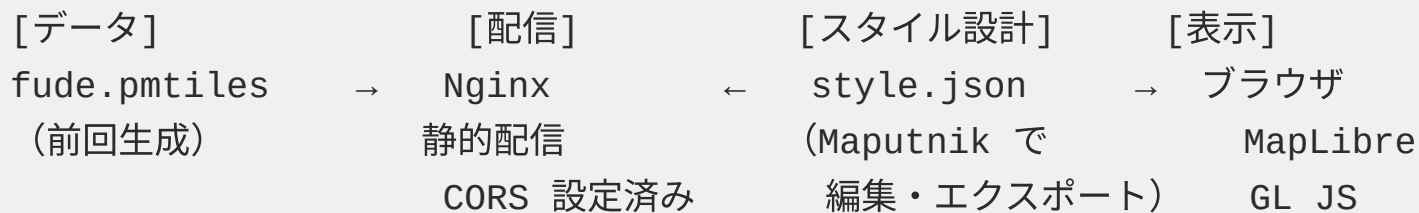
本日の到達目標

- 外部タイルサービスに依存しない地図配信環境を自分で構築できる
- MapLibre Style Spec の構造を理解してスタイルを編集できる
- OSM のデータモデルを理解し、iD エディタで地物を編集できる

前回生成した `fude.pmtiles` を持参・共有してください。

第2回の全体像

今日構築するものの構成



ポイント

- タイルサーバー（Martin 等）は不要。Nginx が静的ファイルを返すだけで動く
- スタイルは Maputnik で編集し `style.json` としてエクスポートする
- MapLibre はスタイル JSON を読んでブラウザ上でレンダリングする

今日の作業はこの3つの接続を順番に確立していく。

1. Nginx による静的配信

PMTiles を自前サーバーから配信する

Nginx のインストール

```
# macOS  
brew install nginx  
  
# Linux (Ubuntu / Debian)  
sudo apt install nginx  
  
# Windows → WSL 内で Linux の手順を実行
```

動作確認：

```
nginx -v  
# nginx/1.x.x  
  
# macOS  
brew services start nginx  
  
# Linux  
sudo systemctl start nginx
```

PMTiles を配信する設定

```
server {  
    listen 80;  
    server_name _;  
    root /var/www/tiles;  
  
    location / {  
        # CORS — MapLibre からのクロスオリジンリクエストを許可  
        add_header Access-Control-Allow-Origin "*";  
        add_header Access-Control-Allow-Methods "GET, OPTIONS";  
        add_header Access-Control-Allow-Headers "Range";  
  
        # Range Request を受け付けると宣言 (PMTiles に必須)  
        add_header Accept-Ranges bytes;  
  
        if ($request_method = OPTIONS) { return 204; }  
        try_files $uri $uri/ =404;  
    }  
}
```

同一オリジンポリシーとは

ブラウザは「どのオリジン（スキーム + ホスト + ポート）から来たリクエストか」を判定し、異なるオリジンへのアクセスをデフォルトでブロックする。

オリジンの定義：スキーム + ホスト + ポート

<code>https://example.com:443</code>	↔	<code>https://example.com:443</code>	→ 同一
<code>http://localhost:3000</code>	↔	<code>http://localhost:8000</code>	→ 別（ポートが違う）
<code>http://localhost:8888</code>	↔	<code>http://localhost:8000</code>	→ 別（同上）

身近な例

React（`localhost:3000`）と Rails API（`localhost:8000`）を開発中に連携させようとするすると CORS エラーが出る。これは意図的な設計で、異なるオリジンからの読み取りをデフォルトで禁止することで、無関係なサイトがAPIを勝手に呼び出せないようにしている。

CORS とは何か・なぜ必要か

Cross-Origin Resource Sharing — 同一オリジンポリシーの例外を許可する仕組み。

サーバーが `Access-Control-Allow-Origin` ヘッダーを返すことで「このリソースは他オリジンから取得してよい」と宣言する。

Maputnik が PMTiles を参照する仕組み

Maputnik はスタイルエディタなので、直接読み込めるのは `style.json` のみ。

PMTiles（タイルデータ）は「どこにあるか（URL）」を Data Sources に指定して参照する。

Maputnik で読み込めるもの → `style.json` (Upload で直接読み込み可)

Maputnik で読み込めないもの → `fude.pmtiles` (URL 参照が必要)

今回 CORS が必要になる場面

`app.maputnik.com` (外部サービス)

→ Data Sources に `http://localhost:8000/fude.pmtiles` を指定

第2回：タイル配信① 自前配信と localhost:8000 は別オリジン

→ ブラウザがブロック → Nginx に CORS 設定が必要

* を許可するリスクと許容できるケース

Access-Control-Allow-Origin: * はすべてのオリジンからのアクセスを許可する。

本来のリスク

ログイン済みのユーザーが悪意あるサイトを開いた場合、そのサイトからユーザーのブラウザ経由でAPIが呼び出される。Cookie や Authorization ヘッダーを使った認証付きリクエストでは、ユーザーの権限で操作・情報取得が行われる危険がある。

悪意あるサイトを開く

- ブラウザが自動的にログイン済みの認証情報を付与してAPIを呼ぶ
- サーバーはユーザー本人からのリクエストとして処理する
- 情報漏洩・意図しない操作が成立する

タイル配信で * が許容できる理由

- 読み取り専用（書き込みAPIではない）
- 認証なし（Cookie・トークンが不要）
- もともと公開する前提のデータ

動作確認

PMTiles をサーバーに配置して Range Request が通るか確認する。

```
sudo mkdir -p /var/www/tiles
sudo cp fude.pmtiles /var/www/tiles/

# Range Request の確認
curl -I -H "Range: bytes=0-511" http://localhost/fude.pmtiles
# HTTP/1.1 206 Partial Content が返れば OK
```

MapLibre からの接続：

```
map.addSource('fude', {
  type: 'vector',
  url: 'pmtiles://http://サーバーIP/fude.pmtiles'
});
```

2. Maputnik でスタイルを設計する

地図の見た目を GUI で編集し style.json としてエクスポートする

MAPUTNIK

Maputnik とは

MapLibre Style Spec を GUI で編集できるオープンソースのスタイルエディタ。
編集結果を `style.json` でエクスポートし、そのまま MapLibre に渡せる。

```
# Web 版（インストール不要・本日はこちらを使用）  
https://app.maputnik.com/
```

ローカル版は Docker で起動できる（`docker run -p 8888:8888 maputnik/editor`）が、本日は Web 版を使用する。

読み込めるファイルと編集できるもの

読み込み方法 (Open から選択)

方法	使い方
From URL	スタイル JSON の URL を直接入力
Upload	ローカルの <code>style.json</code> をアップロード
New Style	白紙のスタイルから始める

編集できるもの

- `sources` : タイルのデータソース (URL・タイル形式)
- `layers` : 色・太さ・透明度・フィルター条件
- `sprite`・`glyphs` : アイコン・フォントの参照先

エクスポート

「Export」で `style.json` をダウンロード。このファイルを MapLibre に渡すだけで動く。

第2回：タイル配信②：自前配信とカートグラフィック

MAPUTNIK — STEP 1

地理院地図ベクタータイルを読み込む

「Open」 → 「From URL」 → 以下を入力：

```
https://gsi-cyberjapan.github.io/gsvectortile-mapbox-gl-js/std.json
```

読み込むと左パネルにレイヤー一覧が展開される。

確認してほしいこと

- レイヤーの数と種類（road / building / railway / boundary など）
- 各レイヤーの `source-layer` 名（後のフィルターで使う）
- Paint プロパティで何が設定されているか

地理院地図ベクタータイルのスタイルは100以上のレイヤーで構成されている。

鉄道レイヤーだけ取り出す

地理院地図ベクタータイルから鉄道だけを抽出してスタイルを変える例。

左パネルで `railway` 系のレイヤーを探して選択 → Paint を変更：

```
{
  "id": "railway-demo",
  "type": "line",
  "source": "v",
  "source-layer": "railway",
  "filter": ["==", "type", "railway"],
  "paint": {
    "line-color": "#e05a00",
    "line-width": 2
  }
}
```

他のレイヤーの目のアイコンをクリックして非表示にすると、鉄道だけが浮かび上がる。

池だけ取り出して地形と重ねる

地理院地図ベクタータイルから水域（池）だけを取り出し、Mapterhorn の地形と重ねる例。

```
{  
  "id": "water-polygon",  
  "type": "fill",  
  "source": "v",  
  "source-layer": "waterarea",  
  "paint": {  
    "fill-color": "#89bdd3",  
    "fill-opacity": 0.8  
  }  
}
```

Mapterhorn（地形 DEM）をソースに追加して terrain を有効化：

```
map.addSource('terrain', {  
  type: 'raster-dem',  
  url: 'pmtiles://https://mapterhorn.example.com/terrain.pmtiles',  
  encoding: 'terrainium'
```

MAPUTNIK — STEP 2

データソースを追加する

右上「Data Sources」 → 「Add New Source」

フィールド	入力値
Source ID	fude
Source Type	Vector（TileJSON / PMTiles）
URL	http://localhost:8000/fude.pmtiles

ローカルの PMTiles を参照するには事前にローカルサーバーが必要：

```
# fude.pmtiles と同じディレクトリで実行
python3 -m http.server 8000
```

MAPUTNIK — STEP 3

レイヤーを追加する

左パネル下部「+ Add Layer」

フィールド	入力値
Layer ID	fude-fill
Layer Type	Fill
Source	fude
Source Layer	fude

Source Layer の名前は **tippecanoe** の **-l** で指定した文字列と一致させる。

MAPUTNIK — STEP 4

色と透明度を設定する

レイヤーを選択 → 「Paint」 タブ

プロパティ	値
fill-color	#6ab04c
fill-opacity	0.5

ズームに応じて変化させる：

fill-opacity の右「+」 → 「Zoom」を選択

Zoom	値
10	0.2
17	0.7

エクスポートして MapLibre に渡す

「Export」 → `style.json` をダウンロード。

```
// 初期化時に渡す
const map = new maplibregl.Map({
  container: 'map',
  style: './style.json'
});

// 後から切り替える
fetch('./style.json')
  .then(r => r.json())
  .then(style => {
    map.once('idle', () => map.setStyle(style));
  });
```

`setStyle()` 後にソース追加する場合は `styledata` より `idle` を使う。

3. MapLibre Style Spec

スタイル JSON の構造とコードとの対応を理解する

MapLibre Style Spec とは

MapLibre GL JS が地図をどのように描画するかを定義する JSON 形式の仕様。
Maputnik が編集・出力する `style.json` はこの仕様に従っている。

主な構成要素：

キー	役割
<code>sources</code>	タイルデータの取得先（URL・形式）
<code>layers</code>	描画レイヤーの定義（色・太さ・フィルター条件）
<code>sprite</code>	アイコン画像の参照先
<code>glyphs</code>	フォントの参照先

仕様は MapLibre 公式で公開されており、Mapbox GL JS と互換性がある。
Maputnik で編集した内容はすべてこの JSON に反映される。

スタイルファイルの構造

```
{
  "version": 8,
  "sources": {
    "fude": { "type": "vector", "url": "pmtiles://./fude.pmtiles" }
  },
  "layers": [
    {
      "id": "fude-fill",
      "type": "fill",
      "source": "fude",
      "source-layer": "fude",
      "paint": {
        "fill-color": "#6ab04c",
        "fill-opacity": ["interpolate", ["linear"], ["zoom"], 10, 0.2, 17, 0.7]
      }
    }
  ]
}
```

スタイルの渡し方：2パターン

	パターン① URL / JSON	パターン② インライン
向いている場面	Maputnik で作ったスタイルを使う	コードで管理したい
レイヤー定義	JSON 側	コード側
PMTiles 直参照	URL に書けば可	<code>addProtocol()</code> が必要
タイルの中身	MVT (PBF)	MVT (PBF) ← 同じ

パターン②（インライン）：

```
const protocol = new pmtiles.Protocol();
maplibregl.addProtocol('pmtiles', protocol.tile);

const map = new maplibregl.Map({
  container: 'map',
  style: {
    version: 8,
    sources: {
      fude: {
        type: 'vector', url: 'pmtiles://./fude.pmtiles' } },
    layers: [{ id: 'fude-fill', type: 'fill', source: 'fude',
```

デモ①

ラズパイタイトルサーバーへの接続体験

会場内の Raspberry Pi 3B+ に接続して、オフライン配信を体感してください。

接続先：（当日アナウンス）

- 同一 Wi-Fi に接続した状態でブラウザからアクセス
- DevTools の Network タブでレスポンスタイムを確認
- ベクタータイトルとラスタータイトルの両方を配信しています

接続先は当日アナウンスします。

デモ① — 補足

ラズパイタイルサーバーの構成

Raspberry Pi 3B+ (OS Lite 64-bit)

- └─ Martin (Rust製タイルサーバー)
- └─ PMTiles (宇部市エリア：ベクター・ラスター)
- └─ Nginx (静的配信・CORS・Range Request 設定済み)
- └─ Tailscale (デモ前の動作確認・SSH用)

ユースケース想定：

- 遭難の多い山岳エリアの山小屋に設置
- 電波が届かない現地でも Wi-Fi 経由でタイル配信
- 登山者のスマートフォンから接続 → オフライン地図として機能

Pi 3B+ は消費電力が低くバッテリー運用も現実的。山小屋・避難小屋への常設を想定したサイズ感。

4. OSM とデータパイプライン

自分で整備したデータをタイル配信に乗せる

OSM エコシステム

OpenStreetMap とは

誰でも編集できる地理空間データベース。

項目	内容
ライセンス	ODbL（Open Database License）
データ形式	ノード・ウェイ・リレーションの3要素
タグ体系	key=value の自由記述（Wiki で標準化）
主な利用形態	タイル配信・データエクスポート・ルーティング

ノード（点）

→ POI・交差点

ウェイ（線）

→ 道路・河川 / 閉じたウェイ → 建物・土地利用

リレーション

→ 複数要素の集合（行政区域・路線など）

OSM エコシステム

iD エディタ デモ（5分）

ブラウザで動作する OSM の標準エディタ。現地調査ツールとしても使える。

1. **POI の追加**：ノードを置いて `amenity=parking` + `capacity=20` を付与
2. **建物の修正**：ウェイの頂点をドラッグして形状を整える
3. **道路の属性編集**： `highway=residential` を確認・変更

現地調査ツールとして

- GPS でノードを打ち、その場で属性を入力して OSM に還元できる
- 駐車場の台数・アクセス路の状態など現地でないと分からない情報の整備に向く
- 収集データは GeoFabrik でエクスポートして自前 GIS に取り込める

編集には OSM アカウントが必要。事前に作成しておくでデモに参加できます。

オフライン地図・PWA への応用

	通信前提	オフライン設計
仕組み	タイルサーバーに都度リクエスト	PMTiles を端末にバンドル
更新頻度	高い（常に最新）	低い（バンドル更新が必要）
適したケース	都市部・屋内	山岳・地下・海上

PWA との組み合わせ

Service Worker で PMTiles をキャッシュ → 初回通信後はオフラインで動作。
現地調査アプリ（POI 入力 + 地図表示）が通信なしで完結する。

第1回からの流れと接続

```
iD エディタで現地編集 → OSM に還元 → GeoFabrik でエクスポート
→ tippecanoe で PMTiles 生成 → Nginx / Martin で配信
```

デモ②

地理院地図ベクターの試行錯誤

地理院地図ベクタータイルのスタイルカスタマイズをソースコードを見ながら解説します。

参考実装：<https://alt9800.github.io/sample-maps/maplibre-catalog/>

詳細な試行錯誤のメモは [2026-06-19/handson/04_gsi_vector/NOTES.md](#) を参照。

まとめ

本日の到達確認

#	確認項目
☐	Nginx で PMTiles を静的配信できた
☐	CORS の設定が必要な理由を説明できる
☐	Maputnik でスタイルを編集して MapLibre に反映できた
☐	スタイルの渡し方（URL 指定 / インライン）の違いを説明できる
☐	OSM のデータモデル（ノード・ウェイ・リレーション）を説明できる
☐	通信前提とオフライン設計のトレードオフを説明できる

第3回（7/3）予告

3D技術① スマホセンサーとAR

キーワード： GeolocationAPI ・ DeviceOrientationEvent ・ A-Frame ・ AR.js ・ WebXR

- スマートフォンの GPS ・ ジャイロ ・ カメラを Web API から利用する
- GeolocationAPI と DeviceOrientationEvent の特性と制約を整理する
- 地理座標に紐づいた AR アプリを実装する
- リモートセンシング由来の観測値をカメラ映像に重ねて表示する